

UNIVERSITY of
BRADFORD

By: Samik Hafeez

(22062361)

Supervisor

Haroon Wahab

Date

06/01/2025

Contents

1. Introduction to the Problem 2
2. Dataset Description and Preprocessing and Libraries used 3
2.1. Dataset Overview 3
2.2. Data Pre-processing 4
2.3. Libraries Used 5
2.4. Code Used for Pre-Processing and Descriptive Analytics, Visualizations 6
3. Design of Experiments for Data Science Solution 10
3.1. Problem Statement 10
3.2. Methodology 10
3.3. Pipeline for the Analysis 11
4. Descriptive Analytics Methods and Visualization 12
4.1. Visualizing the Dataset 12
4.2. Descriptive Analytics Methods 13
5. Selection of Models, their Significance and Hyperparameters 14
5.1. Selected Models: 14
5.2. Code for the Model Creation and Development 15
5.3. Hyperparameters 19
6. Testing and Validation 21
6.1. Testing and Validation Approach. 21
6.2. Code Used for Evaluation 21
6.3. Testing and Validation Results 22
6.4. Evaluation Metrics 27
7. Discussion of Accuracy and Efficiency 28
7.1. Random Forest Classifier: 28
7.2. Support Vector Machine (SVM) 28
7.3. K-Nearest Neighbors (KNN): 29
7.4. Logistic Regression: 29
7.5. Summary of Findings 29
8. Conclusion on the Data Science Solution 31
9. Appendices 31
10. References 34

1. Introduction to the Problem

Breast cancer is the one of the most prominent cancer deaths cause amongst women. The growth of breast cancer is a layered process consisting of various cell types, while prevention continues to be challenge around the world. Diagnosing breast cancer early is among the best strategies to prevent breast cancer (Sun et al, 2017). According to WHO (2024) 2.3 millions women were detected with breast cancer and there were 67 thousand deaths worldwide. The greater risk of developing breast cancer in among female gender that estimates to around 99% with only 0% to 1% chance among the male gender.

Over the recent decades, Machine learning techniques have broadly been applied for intelligent healthcare systems, particularly to diagnose and prognose the breast cancer (Yue et al, 2018). The main purpose that this report aims to fulfil is to predict the diagnosis of the breast cancer as benign or malignant, using a range of analytical features attained from imaging data.

This report emphasizes on the application of machine learning methods to anticipate the presence of breast cancer as benign or malignant employing Breast Cancer Wisconsin (Diagnostic) dataset. The dataset consists of variety of features that are processed from a digitalized picture of a fine needle aspirate(FNA) of a breast mass that describe the attributes of a cell nuclei that exist in the image (Wolberg et al, 1993). The project is aimed to build a predictive model capable of assisting medical professionals in diagnosing the malignant tumours in advance, enhancing patient results, and assisting quicker intervention.

In this report, various machine learning algorithms will be explored, consisting Logistic Regression, Support Vector Machine, Random Forest Classifier, and K-Nearest Neighbors (KNN). The models are expected to be trained on the selected portion of the data, will be assessed using cross-validation, and other metrics like precision, recall, and F1-score, AUC-ROC and Confusion Matrix will be used to assess their performance based on accuracy.

2. Dataset Description and Preprocessing and Libraries used

2.1. Dataset Overview

The Breast Cancer Wisconsin (Diagnostic) dataset is available at UC Irvine Machine Learning Repository to be used publicly and comprises of 569 breast cancer observations. The dataset contains 30 features that represent the characteristics of cell nuclei present in the samples. These features have been obtained from a digitized image of fine needle aspirate (FNA) of a breast mass (Wolberg et al, 1993). They include:

- Radius: The mean of the stretch from the middle to locations on the perimeter.
- Texture: The standard devotion of grey-scale values.
- Perimeter: The perimeter of the tumour.
- Area: The area of the tumour
- Smoothness: The native deviation within the radius distances.
- Compactness: The square of perimeter divided by the area takeaway 1.
- Concavity: The severity of concave parts of the contour.
- Symmetry: The symmetry of the tumour.
- Fractal Dimension: The computation of the complication of the boundary of the tumour.

  (Wolberg et al, 1993)

Each feature has three measurements: mean, standard error and worst that are presented as 1,2, and 3 in the dataset respectively. The measurement assists in obtaining the irregularities for the tumour's attributes. For example, texture1 gives the average texture of the tumour, while texture3 acquires the extreme observed texture in the dataset.

The target variable in the dataset is the target, which contains two classed categories namely:

- Malignant (M): Cancerous tumour that expands to others parts of the body.
- Benign (B): Non-cancerous tumour that does not spread.

2.2. Data Pre-processing

Before applying machine learning models to the dataset, key pre-processing steps were carried out to guarantee that the data was cleaned, completed and suitable for the analysis. The steps are as follows:

- **Handling Irrelevant and Missing Columns**
    - The Id and Diagnosis columns were removed as they were completely empty and did not provide assistance towards the analysis
    - An Index column was added for the improved data organization and access during processing.
- **Target Column Conversion**

  The target column was transformed from categorial values M and B to numeric values 1 and 0 respectively to ensure functionality with machine learning models.
- **Handling Outliers**

  Outliers were identified and eliminated using the Interquartile Range (IQR) method. Features with excessive values outside the IQR range from first and third quartiles were removed to improve the dataset's quality and reduce noise. There were more outliers but removing them would result in less data for model training so the dataset was kept as it is.
- **Normalization**

  The dataset was normalized using the Min-Max Normalization method to scale all feature values to ensure the values are between 0 and 1 that

guarantees that feature and uniform for model training.

The formula used:

$$Xnorm = (X-Xmin) / (Xmax-Xmin)$$

- **Feature Selection**

  All the features were retained; However, the dataset was reduced due to the removal of the outliers.

- **Addressing Missing Values**

  The dataset did not include any missing values, but it was still checked for missing values. The imputation techniques were not employed.

- **Data Splitting**

  The dataset was divided into training and testing sets that accounts for 80% and 20% of dataset for training and testing data respectively to assess the model performance on unused data.

2.3. Libraries Used

- **Pandas:** It is a python library that is used to work with datasets. It is used for analysis, cleaning, manipulation and exploration of the data. (w3schools,2025)
- **Seaborn:** It is a Python visualization library used for data and is based on matplotlib. It provides a responsive platform to draw gorgeous and informative statistical figures (Waskom, 2024)
- **Matplotlib:** It is a complete library used for making static, collaborative and dynamic visualizations in python. (Matplotlib,2024)
- **NumPy:** It is an open-source library used in Python for scientific computing programming tasks. It is used for advanced mathematical operations that include matrices, arrays, and multidimensional arrays.(Bigelow,2025)
- **Scikit learn/sklearn:** It is a machine learning library in python that contains numerous libraries exceptional for statistical modelling for classification, clustering, dimensional reduction and regression. It is also used for training,

testing, fitting, selection, validation and of the machine learning models.(Jain,2024)

- **SciPy:** It is an open-source library used to solve scientific and mathematical problems in python. It allows the manipulation and visualization of advanced data with broad range.(Johari,2019)

2.4. Code Used for Pre-Processing and Descriptive Analytics, Visualizations

The Code used will be provided below: The results will be provided in Appendix A.

- **Descriptive Analytics and Pre-processing**

```
# Provide the number of Benign and Malignant cases are there in the dataset.
counts = df.groupby('target').size()
print('Number of Benign and Malignant cases')
print(counts)

# Provide the description of the Dataset
print('Description of the dataset')
print(df.describe())

# Displaying the shape of the dataset
print('Dataset Shape:')
print(df.shape)

# Display the DataFrame data types
print('DataTypes of the Dataset.')
print(df.dtypes)

# Displaying the DataFrame information
df.info()#

# Removing the Diagnosis row along with its columns
del df['Diagnosis']

# Removing the ID row along with its column
del df['ID']

# Displaying the columns
print('Displaying the columns')
print(df.columns)

# Displaying the values of the DataFrame
print('Displaying the values of the DataFrame')
print(df.values)
```

```
# Checking for the null value in the DataFrame
print('Checking for the null values in the DataFrame')
print(df.isnull())

# Counting the total number of null values in the DataFrame

print('checking for the null values in the DataFrame')
print(df.isnull().sum())




print(df.head())
```

```
import numpy as np
from collections import Counter

def detect_outliers(train_data, n, features):
    outlier_indices = []
    for col in features:
        # 1st quartile (25%)
        Q1 = np.percentile(train_data[col], 25)
        # 3rd quartile (75%)
        Q3 = np.percentile(train_data[col], 75)
        # Interquartile range (IQR)
        IQR = Q3 - Q1
        outlier_step = 1.5 * IQR
        outlier_list_col = train_data[(train_data[col] < Q1 - outlier_step) | (train_data[col] > Q3 + outlier_step)].index
        outlier_indices.extend(outlier_list_col)

    outlier_indices = Counter(outlier_indices)
    # Filter out the outliers that appear more than 'n' times
    multiple_outliers = [k for k, v in outlier_indices.items() if v > n]

    return multiple_outliers

# Exclude the 'target' column from the list of features
list_atributes = new_df.drop('target', axis=1).columns

# Detect outliers where an index appears more than once in the outlier list
Outliers_to_drop = detect_outliers(new_df, 2, list_atributes)

# Print or use the detected outliers
print("Outliers to drop:", Outliers_to_drop)
```

- Code for Visualizations

```
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
sns.countplot(x='target',data=df,palette='viridis')
plt.title('Target class distribution')
plt.show()
```

```
new_df=df.copy()
new_df['target']=new_df['target'].map({'M': 1, 'B': 0})
plt.figure(figsize=(30,30))
sns.heatmap(new_df.corr(),annot=True,cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Histograms (excluding first and last columns)
for i, dataset in enumerate([new_df], 1):
    dataset.iloc[:, 1:-1].hist(bins=20, figsize=(30, 30))  # Exclude first and last columns
    plt.suptitle(f"Dataset{i} Feature Distributions")
    plt.show()

# Heatmaps (excluding first and last columns)
for i, dataset in enumerate([new_df], 1):
    plt.figure(figsize=(30,30))
    sns.heatmap(df.iloc[:, 1:-1].corr(), annot=True, cmap="coolwarm")  # Exclude first and last columns
    plt.title(f"Dataset Correlation Heatmap")
    plt.show()
```

```python
import seaborn as sns
import matplotlib.pyplot as plt

# Strip plots (excluding first and last columns)
numeric_features = df.columns[1:-1]  # Exclude first and last columns

# Create a grid of subplots (3 rows and 10 columns)
rows = 10
cols = 3
fig, axes = plt.subplots(rows, cols, figsize=(20, rows * 6))

# Flatten axes to make indexing easier
axes = axes.flatten()

# Loop through each feature and plot on the respective subplot
for idx, feature in enumerate(numeric_features):
    ax = axes[idx]  # Select the appropriate axis from the flattened axes
    sns.stripplot(x='target', y=feature, data=df, jitter=True, palette="Set1", alpha=0.7, ax=ax)
    ax.set_title(f"Strip Plot: {feature} vs target")
    ax.set_xlabel('Target')
    ax.set_ylabel(feature)

# Adjust layout for better presentation
plt.tight_layout()
plt.show()
```

Python

```python
# Visualize the data
sns.pairplot(new_df, hue='target', vars=['radius1', 'texture1', 'perimeter1', 'area1', 'smoothness1', 'compactness1', 'concavity1', 'concave_points1', 'symmetry1','fractal_dimension

# Set the title for the entire figure
plt.suptitle('Scatter matrix of target and model features', fontsize=16)
```

```python
import matplotlib.pyplot as plt
import seaborn as sns
#Code for Boxplot

# Create a figure with custom size
fig = plt.figure(figsize=(16, 20))

# Iterate over the first 30 columns of your DataFrame
for i, b in enumerate(new_df.columns[1:31]):
    # Create axes object for position i (adjusting for subplot index)
    i += 1
    ax = fig.add_subplot(6, 5, i)  # Create a subplot with 6 rows and 5 columns

    # Check if both df_n and df_y contain data for this column
    if b in df_n.columns and b in df_y.columns:
        # Plot the boxplot for comparison of malignant and benign
        ax.boxplot([df_n[b].dropna(), df_y[b].dropna()], vert=True, patch_artist=True, widths=0.5)
        # Set the title and labels for each subplot
        ax.set_title(b)
        ax.set_xticklabels(['Malignant', 'Benign'])
        ax.set_ylabel('Values')
        ax.set_xlabel('Diagnosis')
    else:
        # If a column doesn't exist in either df_n or df_y, print a warning
        print(f"Warning: Column '{b}' not found in both df_n and df_y.")

# Apply Seaborn's whitegrid style for aesthetics
sns.set_style("whitegrid")

# Adjust layout to prevent overlap of subplots
plt.tight_layout()

# Show the final plot with legends
plt.legend(["Malignant", "Benign"], loc='upper right', bbox_to_anchor=(1.1, 1))
plt.show()
```

3. Design of Experiments for Data Science Solution

3.1. Problem Statement

The problem is categorized as a binary classification task. The aim is to classify the diagnosis of the breast cancer as either Benign 0 or Malignant 1 based on the features available in the Breast cancer Wisconsin dataset based on this report uses binary classification as we are predicting two outcomes only.

3.2. Methodology

The analysis methodology used is CRISP-DM framework, that focuses on repetitive development. This report is aimed for classification, as the target variable is binary. CRISP-DM is a repeated process that provides a structured strategy to plan, organize and implement a data mining project (Chumbar,2023). There are six stages to CRISP-DM that can be visualized in the figure 1. Whereas only 5 for them are used for the project deployment is not used.

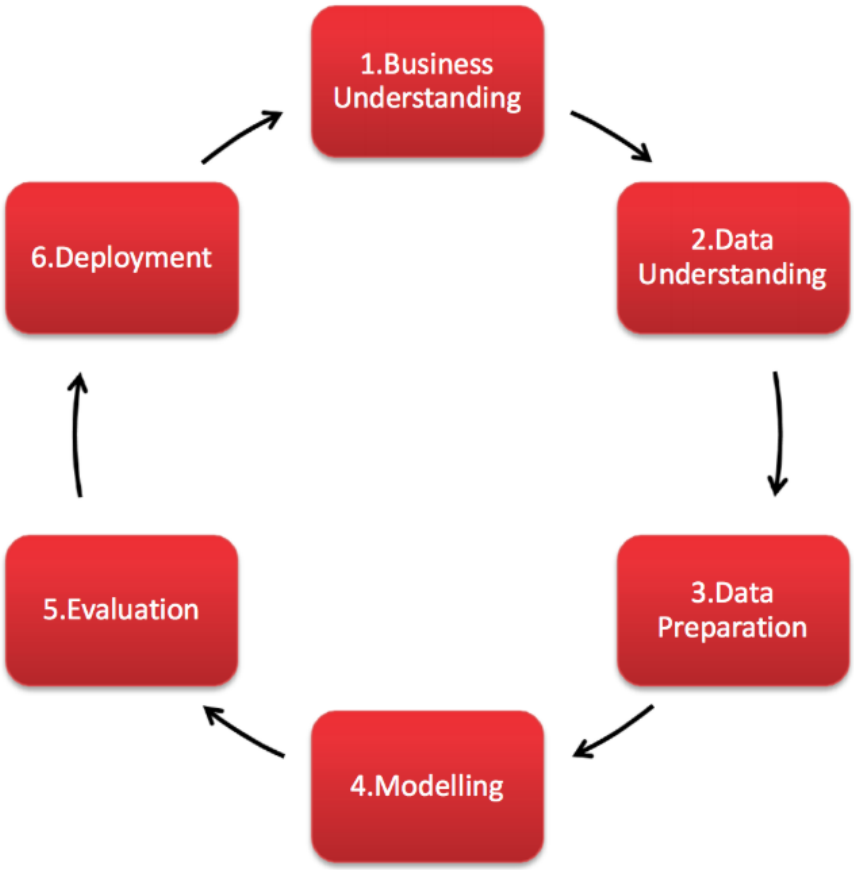


Figure 1 Six stage of CRISP-DM (SV Eurpoe,2025)

3.3. Pipeline for the Analysis

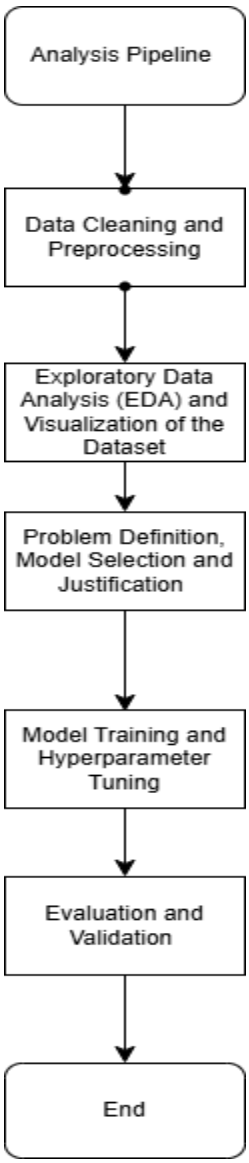


Figure 2 Pipeline of the Analysis (draw.io,2024)

4. Descriptive Analytics Methods and Visualization

4.1. Visualizing the Dataset

Exploratory Data Analysis (EDA) is a crucial step in understanding the structure of the dataset and the relationships between the features. Numerous visualization techniques were employed to gather intuitions into the data and ensure that the features were explored adequately before training the model. The dataset was visualized using the following:

- **Target Variable Distribution**

  A bar chart was created to visualize the distribution of benign and malignant cases, showcasing the class imbalance.

- **Correlation Heatmap**

  Utilized Seaborn to form a heatmap to understand the correlation between the features assisting in facilitating towards figuring out feature significance.

- **Histogram**

  Generated histograms for each feature to examine data allocation and identify the skewness.

- **Pairwise Feature Relationships**

  Seaborn pair plots were used to examine the relationships between the significant features, contrasted with the target variable.

- **Box Plots**

  Plotted the box plot to explore the key features and gather insights between features to identify differences among features.

- **Scatter Plots**

  Mapped PCA and t-SNE Scatter Plots to monitor the separation between the malignant and benign cases and provided visualization for data clustering patterns.

- **ROC Curve**

  For evaluation of the model, ROC curves were used to showcase balance between the sensitivity and accuracy for trained models

- **Swarm and Strip Plots**

  Swarm and strip plots were used to illustrate data distributions for individual features compared to the target variable that highlighted class-specific outliers and trends.

- **Probability Density Function (PDF) and Cumulative Distribution Function (CDF) Curves**

  Mapped PDF and CDF curves to identify the key features and explore their distributions. These curves help recognize skewness, kurtosis, and other statistical descriptions leading towards features explanation.

4.2. Descriptive Analytics Methods

The following descriptive analytics methods were used to explore and understand the dataset:

- **Dataset Overview**

  The dataset was first examined using methods .shape, .dtypes , describe(), and info(), .tail(), .head() function to gather insights of data structure, data types, and statistical figures. The dataset initially contained 569 rows and 34 columns, that contained numerical and categorical data.

- **Missing Data and Null Values**

  To discover missing data in the dataset, the .isnull() methods was used. The total number of null values was checked using .isnull().sum() function to ensure the data is free of missing values and were detected before starting the analysis.

- **Target Variable Distribution:**

  The distribution of the target variable that is dispersed as M for Malignant and B for Benign and was analysed using .groupby('target').size(). This assisted in identifying how the data is distributed. The dataset had 357 Benign and 212 Malignant cases.

- **Feature Summary Statistics**

  Summary statistics for every feature was created using .describe() function, presenting intuitions into the central tendencies (mean, median), range , (min,max) and spread (standard deviation) of the features instances. This provided the understanding of the attributes of every feature in the dataset.

- **Feature Removal**

  Unnecessary columns like Id and Diagnosis were removed from the dataset. Ensuring that only useful features were preserved for modelling purposes, enhancing model effectiveness and explainability.

- **Normalization of Data**

  To prepare the dataset for modelling for the Min-Max normalization was applied to scale the features, confirming that all features are within a uniform range of (0 to 1). This helped refraining features with greater numerical ranges from affecting the model responses.

5. Selection of Models, their Significance and Hyperparameters

For the analysis, Logistic Regression, Support Vector Machine (SVM), Random Forest Classifier, and K-Nearest Neighbors (KNN) were utilized to assess which model would be most suitable for the Breast Cancer Wisconsin dataset. The above-mentioned models are selected due to their suitability in binary classification tasks, aimed at detecting whether a tumour is benign or malignant based on the provided features.

5.1. Selected Models:

- **Logistic Regression.**

Definition: Logistic regression makes the estimation about the probability of event occurrence based on provided independent variable data set. (IBM, n.d.)

Why it is used:

- **Explainability:** It provides easy explainable results, ideal for small datasets.
- **Baseline Model:** Logistic Regression is sometimes considered baseline model because of its simplicity.
- **Probabilistic Outcome:** It gives probability of an outcome alongside the prediction of class, which can be useful in binary classification.

- Support Vector Machine (SVM)

Definition: Support Vector Machine is a durable ML algorithm broadly used for classification especially linear and random classification. It can also be used for regression and outliers detection.(Geeksforgeeks,2024)

Why it is used:

- **Efficient for Great Dimensions:** It works exceptional well with extreme-dimensioned data, like the Breast Cancer Wisconsin dataset, that consist of many features.
- **Strong against Overfitting:** SVM attempts to increase the boundaries between classes, making it strong against overfitting compared to other algorithms.

- Random Forest Classifier

Definition: A random forest is a meta predictor that mounts numerous decision tree classifier on different sub-samples of dataset and utilizes the averaging to improve the prediction accuracy and limits the overfitting.(scikitlearn,2024)

Why it is used:

- **High Accuracy:** Random Forest Classifier usually outperforms others models when accuracy is compared, notably on organized data like Breast Cancer Wisconsin dataset.
- **Dealing Complex Data:** Random Forest is tough and deals complex data as well as numerous features.
- **Significant Features:** It can evaluate value of each feature, providing

intuitions into the features that impact the outcomes.

- **K-Nearest Neighbors (KNN)**

  Definition: KNN is a simple supervised machine learning algorithms used for the the classification and regressions tasks. It rely on an idea that instances nearest to the specified data point are the same instances in the data sets. They can classify the unseen points based on the values of the nearest accessible points. (w3schools,2025)

  Why it is used:

  - **Easy to apply:** KNN is easy to grasp and apply, making it ideal for comparison.
  - **Non-parametric**: It does not assume regarding the inherited data distribution, which is useful for certain types of data.
  - **Useful for Small Datasets**: KNN performs exceptionally well for small datasets and is sensitive to the native structure of the data.

5.2. Code for the Model Creation and Development

```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train,y_test = train_test_split(X,y,test_size = 0.20, random_state = 42)

X_train = X_train.T
X_test= X_test.T
y_train = y_train.T
y_test = y_test.T

print("x train: ", X_train.shape)
print("x test: " , X_test.shape)
print("y train:", y_train.shape)
print("y test: ", y_test.shape)
```

```
train:  (30, 388)
test:  (30, 98)
train: (388,)
test:  (98,)
```

```python
def initialize_weights_and_bias(dimension):
    w = np.full((dimension,1),0.01)
    b = 0.0
    return w,b
```

```python
# z = np.dot(w.T, x_train)+b

def sigmoid(z):
    y_head = 1/ (1 + np.exp(-z))
    return y_head
```

```python
def forward_backward_propagation(w,b,X_train,y_train):
    z = np.dot(w.T,X_train) + b
    y_head = sigmoid(z)
    y_head = np.clip(y_head, 1e-8,1 - 1e-8)
    loss = y_train * np.log(y_head) - (1- y_train) * np.log(1-y_head)
    # X_train.shape[1] is for scaling
    cost= (np.sum(loss)) / X_train.shape[1]

    # backward propagation
    derivative_weight = (np.dot(X_train, ((y_head - y_train).T))) / X_train.shape[1]
    derivative_bias = np.sum(y_head-y_train)/ X_train.shape[1]
    gradients = {"derivative_weight": derivative_weight,
                 "derivative_bias": derivative_bias}
    return cost, gradients
```

```python
def update(w,b,X_train,y_train, learning_rate,number_of_iteration):
    cost_list = []
    cost_list = []
    cost_list2 = []
    index = []

    # updating(learning) parameters is number_of_iterations times
    for i in range(number_of_iteration):
        #Make forward and backward propagation and find cost and gradients
        cost, gradients = forward_backward_propagation(w,b,X_train,y_train)
        cost_list.append(cost)

        # lets update
        w = w - learning_rate * gradients["derivative_weight"]
        b = b - learning_rate * gradients["derivative_bias"]
        if i % 10 == 0:
            cost_list2.append(cost)
            index.append(i)
            print("Cost after iteration {}:{}".format(i,cost))
    # update(learn) parameters weights and bias
    parameters = {"weight":w, "bias":b}
    plt.plot(index,cost_list2)
    plt.xticks(index,rotation='vertical')
    plt.xlabel("Number of Iteration")
    plt.ylabel("Cost")
    plt.show()
    return parameters, gradients,cost_list
```

```python
def predict(w,b,X_test):
    # X_test is a input for forward propagation
    z = sigmoid(np.dot(w.T,X_test)+b)
    Y_prediction = np.zeros((1,X_test.shape[1]))

    # if z is bigger than 0.5, our prediction is sign one (y_head = 1),
    # if z is smaller than 0.5, our prediction is sign zero (y_head = 0)
    Y_prediction[0,z[0,:]> 0.5] = 1
    return Y_prediction
```

```
# Load your dataset (Assuming X, y are the features and target variables)
# X, y = load_data()  # Replace with actual data loading method
# For the sake of this example, we will use sklearn's load_breast_cancer dataset
# Splitting data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scaling the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ============================ Random Forest Classifier ============================

# Initialize Random Forest Classifier
rf_classifier = RandomForestClassifier(random_state=42)

# Fit the model
rf_classifier.fit(X_train_scaled, y_train)

# Make predictions
rf_train_pred = rf_classifier.predict(X_train_scaled)
rf_test_pred = rf_classifier.predict(X_test_scaled)

# Calculate accuracy
rf_train_accuracy = accuracy_score(y_train, rf_train_pred) * 100
rf_test_accuracy = accuracy_score(y_test, rf_test_pred) * 100

print(f"Random Forest Classifier - Train Accuracy: {rf_train_accuracy:.2f}%")
print(f"Random Forest Classifier - Test Accuracy: {rf_test_accuracy:.2f}%")
```

```python
# Initialize Support Vector Machine
svm_classifier = SVC(probability=True, kernel='linear', random_state=42)

# Fit the model
svm_classifier.fit(X_train_scaled, y_train)

# Make predictions
svm_train_pred = svm_classifier.predict(X_train_scaled)
svm_test_pred = svm_classifier.predict(X_test_scaled)

# Calculate accuracy
svm_train_accuracy = accuracy_score(y_train, svm_train_pred) * 100
svm_test_accuracy = accuracy_score(y_test, svm_test_pred) * 100

print(f"SVM Classifier - Train Accuracy: {svm_train_accuracy:.2f}%")
print(f"SVM Classifier - Test Accuracy: {svm_test_accuracy:.2f}%")

# ============================ K-Nearest Neighbors (KNN) ============================

# Initialize K-Nearest Neighbors
knn_classifier = KNeighborsClassifier(n_neighbors=5)

# Fit the model
knn_classifier.fit(X_train_scaled, y_train)

# Make predictions
knn_train_pred = knn_classifier.predict(X_train_scaled)
knn_test_pred = knn_classifier.predict(X_test_scaled)

# Calculate accuracy
knn_train_accuracy = accuracy_score(y_train, knn_train_pred) * 100
knn_test_accuracy = accuracy_score(y_test, knn_test_pred) * 100

print(f"KNN Classifier - Train Accuracy: {knn_train_accuracy:.2f}%")
print(f"KNN Classifier - Test Accuracy: {knn_test_accuracy:.2f}%")
```

```python
# Initialize Support Vector Machine
svm_classifier = SVC(probability=True, kernel='linear', random_state=42)

# Fit the model
svm_classifier.fit(X_train_scaled, y_train)

# Make predictions
svm_train_pred = svm_classifier.predict(X_train_scaled)
svm_test_pred = svm_classifier.predict(X_test_scaled)

# Calculate accuracy
svm_train_accuracy = accuracy_score(y_train, svm_train_pred) * 100
svm_test_accuracy = accuracy_score(y_test, svm_test_pred) * 100

print(f"SVM Classifier - Train Accuracy: {svm_train_accuracy:.2f}%")
print(f"SVM Classifier - Test Accuracy: {svm_test_accuracy:.2f}%")

# ============================ K-Nearest Neighbors (KNN) ============================

# Initialize K-Nearest Neighbors
knn_classifier = KNeighborsClassifier(n_neighbors=5)

# Fit the model
knn_classifier.fit(X_train_scaled, y_train)

# Make predictions
knn_train_pred = knn_classifier.predict(X_train_scaled)
knn_test_pred = knn_classifier.predict(X_test_scaled)

# Calculate accuracy
knn_train_accuracy = accuracy_score(y_train, knn_train_pred) * 100
knn_test_accuracy = accuracy_score(y_test, knn_test_pred) * 100

print(f"KNN Classifier - Train Accuracy: {knn_train_accuracy:.2f}%")
print(f"KNN Classifier - Test Accuracy: {knn_test_accuracy:.2f}%")
```

5.3. Hyperparameters

For each model, Grid search (definition) was used with cross validation to achieve the optimal performance for the dataset.

- Logistic Regression

```
# Set up the parameter grid for tuning
param_grid_logreg = {
    'C': [0.001, 0.01, 0.1, 1, 10],  # Regularization strength
    'solver': ['liblinear', 'saga'],  # Solvers for optimization
    'max_iter': [100, 200, 300],  # Maximum number of iterations
    'penalty': ['l1', 'l2']  # Regularization techniques
}
```

  - **C:** Optimal value: 0.1
  - **Solver:** liblinear
  - **max_iter:** Optimal value: 100
  - **penalty:** Optimal value: l1

- Support Vector Machine (SVM)

```
param_grid_svm = {
    'C': [0.1, 1, 10],          # Focused regularization parameters
    'kernel': ['linear', 'rbf'], # Simpler kernels
    'gamma': ['scale']           # Default kernel coefficient
}
```

  - **C:** Optimal value:1.
  - **Gamma:** Optimal value: scale.
  - **Kernel:** Specifies the kernel type to be used in the algorithm. Optimal value: rbf.

- **Random Forest Classifier**

```
param_grid = {
    'n_estimators': [50, 100, 200],  # Number of trees
    'max_depth': [None, 10, 20, 30],  # Depth of each tree
    'min_samples_split': [2, 5, 10],  # Minimum samples required to split a node
    'min_samples_leaf': [1, 2, 4],  # Minimum samples required at a leaf node
    'bootstrap': [True, False],  # Whether to use bootstrap samples
    'max_features': ['sqrt', 'log2', None]  # Corrected value for max_features
}
```

  - **Bootstrap:** Optimal Value:False
  - **max_depth:** Optimal Value: None
  - **max_features:** Optimal Value: sqrt
  - **min_sampes_leaf:** Optimal Value 1
  - **min_samples_split:** Optimal Value 2
  - **n_estimators:** Optimal Value 50

- **K-Nearest Neighbors (KNN):**

```
# Set up the parameter grid for tuning
param_grid_knn = {
    'n_neighbors': [3, 5, 7, 9],  # Number of neighbors to consider
    'weights': ['uniform', 'distance'],  # Uniform or distance-based weighting
    'p': [1, 2]  # Manhattan distance (p=1) or Euclidean distance (p=2)
}
```

  - **Number of Neighbors (n_neighbors):** Optimal value:5.
  - **P:** The power parameter for the Minkowski metric. Optimal value: 1 (Manhattan distance).

- **Weights:** Optimal value: uniform.

6. Testing and Validation

To evaluate the performance of the models LR,SVM,RFC, and KNN. Thorough testing and validation were performed using suitable evaluation metrics. These metrics provide a complete understanding of models' precision, accuracy, F1-score,recall and overall strength.

6.1. Testing and Validation Approach.

- **Train-test Split:** The dataset was split into training and testing sets as 80-20 split respectively. The training set was used for the training of the model, while testing set was used for the evaluation of the model response to unseen data.
- **Cross Validation:** A cross-validation approach with 3 to 5 folds was utilized to confirm that the model is robust and reduce the risk of overfitting. This approach divides the data in numerous subsets to train and validate the model repeatedly.

6.2. Code Used for Evaluation

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score, roc_curve, auc, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Initialize Logistic Regression model
lr_classifier = LogisticRegression(max_iter=1000, random_state=42)

# Train the Logistic Regression model on the training data
lr_classifier.fit(X_train, y_train)

# Function to calculate and print evaluation metrics
def evaluate_model(model, X_test, y_test):
    # Predicting the labels
    y_pred = model.predict(X_test)

    # Precision, Recall, F1-Score
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # AUC-ROC
    y_prob = model.predict_proba(X_test)[:, 1]  # Probabilities for the positive class
    auc_score = roc_auc_score(y_test, y_prob)

    # Print evaluation metrics
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1-Score: {f1:.4f}")
    print(f"AUC-ROC Score: {auc_score:.4f}")
```

```
    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.classes_)
    disp.plot(cmap=plt.cm.Blues)
    plt.title(f'Confusion Matrix for {model.__class__.__name__}')
    plt.show()

    # Plot ROC Curve
    fpr, tpr, thresholds = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)

    plt.figure()
    plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC Curve for {model.__class__.__name__}')
    plt.legend(loc="lower right")
    plt.show()

# Assuming other models (svm_classifier, knn_classifier, rf_classifier) are already trained

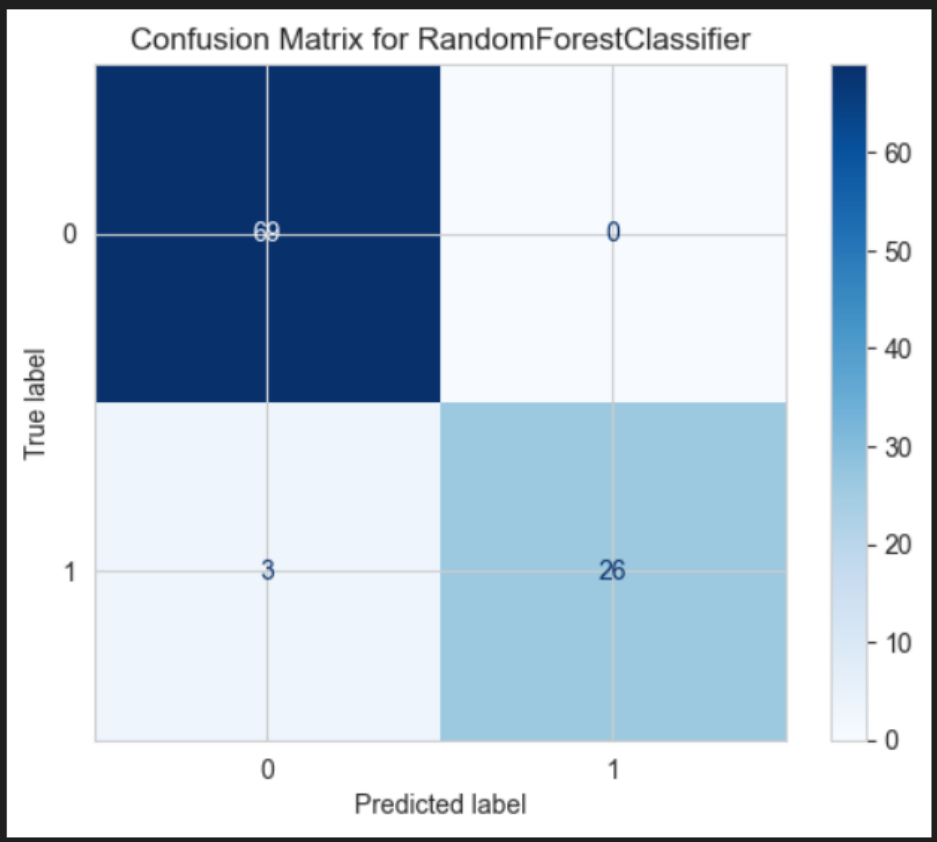
models = [svm_classifier, knn_classifier, rf_classifier, lr_classifier]
model_names = ['SVM', 'KNN', 'Random Forest', 'Logistic Regression']

for model, name in zip(models, model_names):
    print(f"Evaluation for {name}:")
    evaluate_model(model, X_test, y_test)
    print("\n" + "-"*50 + "\n")
```

6.3. Testing and Validation Results

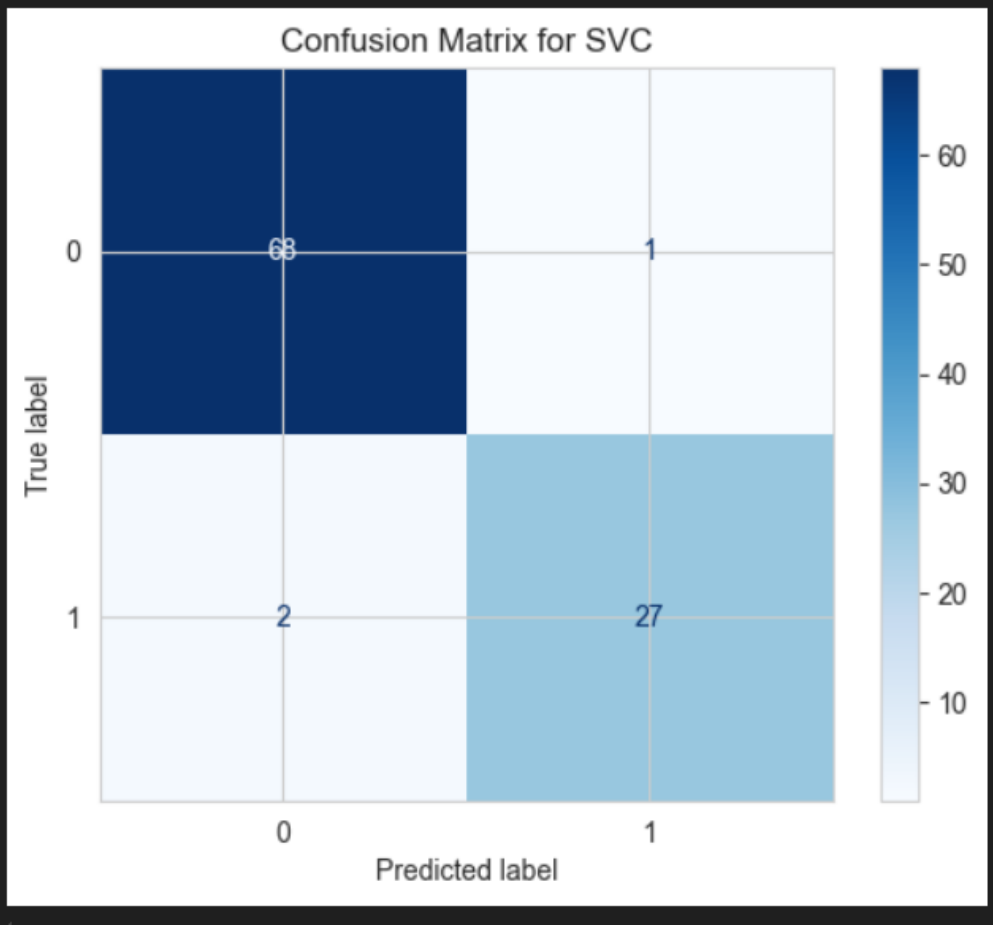
- Random Forest Classifier

```
Evaluation for Random Forest:
Precision: 1.0000
Recall: 0.8966
F1-Score: 0.9455
AUC-ROC Score: 0.9850
```



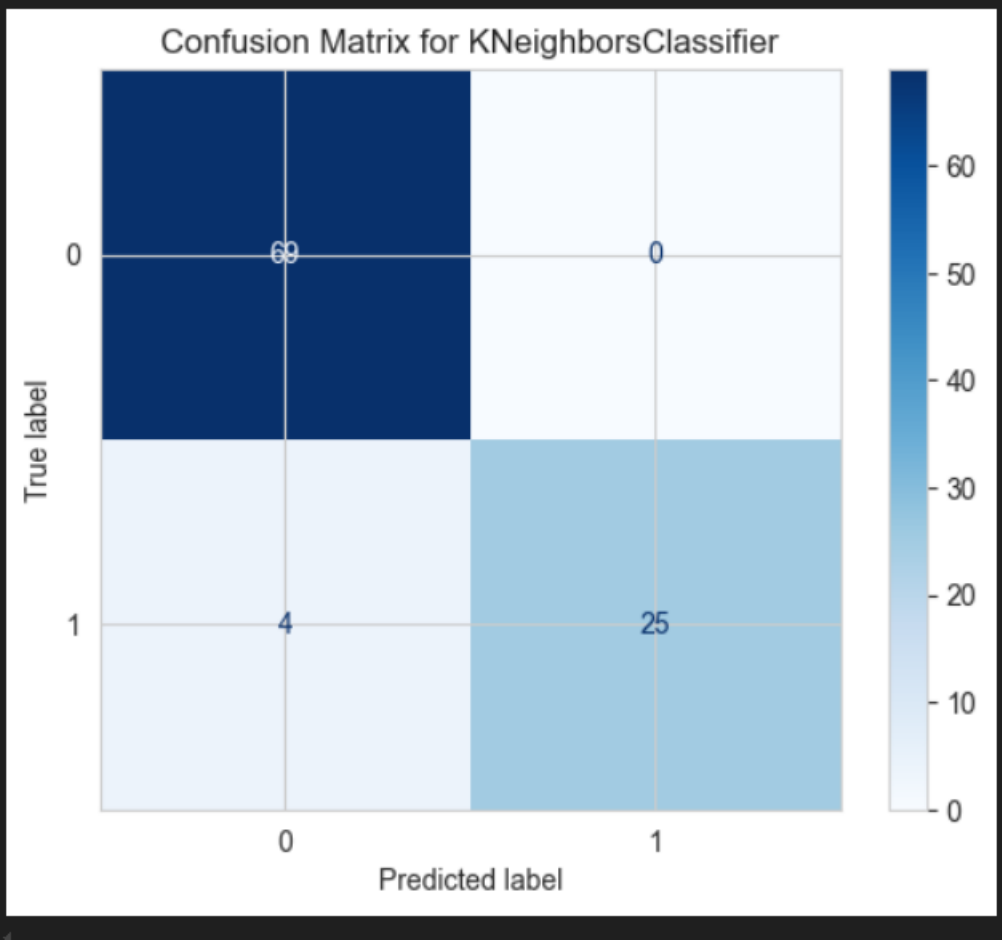
- Support Vector Machine (SVM)

```
Evaluation for SVM:
Precision: 0.9643
Recall: 0.9310
F1-Score: 0.9474
AUC-ROC Score: 0.9865
```



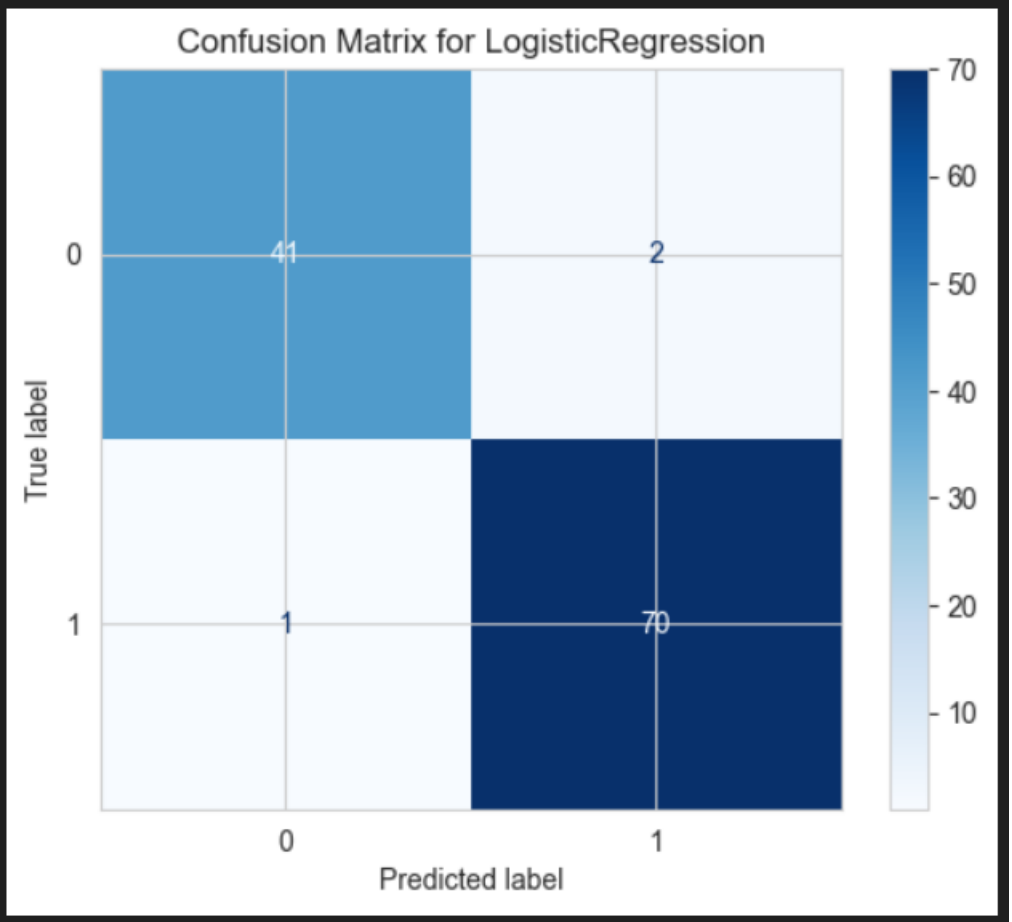
- K-Nearest Neighbors (KNN)

```
Evaluation for KNN:
Precision: 1.0000
Recall: 0.8621
F1-Score: 0.9259
AUC-ROC Score: 0.9783
```



- Logistic Regression

```
Evaluation for Logistic Regression:
Precision: 0.9722
Recall: 0.9859
F1-Score: 0.9790
AUC-ROC Score: 0.9974
```



6.4. Evaluation Metrics

- **Accuracy**

  Calculates the ratios of correctly classified occurrences out of total instances. It provides a standard understand of the model's performance. Accuracy can be defined as:

  Accuracy = TP+TN/TP+ TN+FP+FN (Google Developer,2024).

  Where what TP,TN,FP,FN stand for can be visualized in Figure 2.

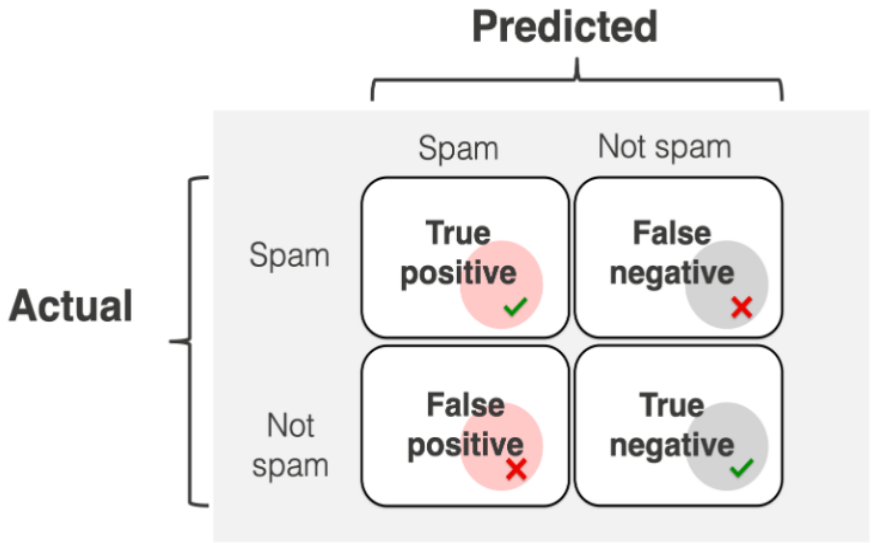


Figure 3 Confusion Matrix (Evidently AI,2024)

- **Precision**

  The percentage of correctly predicted by the total number of occurrences of the model forecasted as positive.

  Precision = TP / TP+FP (Evidently AI,2024)

- **Recall**

  The True positive rate or the percentage of all the definite positives that were categorized accurately as positives.

  Recall = TP/TP+FN (Google Developers,2024).

- **F1-Score**

  F1-Score is an evaluation metric that evaluates the predictable ability of a

model by exploring its performance on each class separately oppose to accuracy that considers the overall performance. (Logunova,2023).

- **AUC-ROC**
- Area Under the Curve- Receiver Operating Characteristics Curve is a performance measure where ROC is a probability curve and AUC is the degree (Narkhede,2018).
- **Confusion Matrix**

  A confusion matrix is a matrix that sums up the performance of a machine learning model on a set of a test data. (Geeksforgeeks,2024)

7. Discussion of Accuracy and Efficiency

The models were evaluated based on their accuracy on both training and test sets what are as follows:

7.1. Random Forest Classifier:

- Train Accuracy: 100%
- Test Accuracy: 96.49%
- **Explanation:** Random Forest has the perfect accuracy for the training set, that shows that models fits the training data flawlessly. However, this indicates that the model is overfitting; Meaning that the model has learned too well from the training data and might not perform well on unseen data.
- **Efficiency:** Random Forest is solid but costs more computational power opposed to simpler models. It grouping nature makes it slow, that is not ideal for large dataset.

7.2. Support Vector Machine (SVM)

- Train Accuracy: 98.68%
- Test Accuracy: 95.61%

- **Explanation:** SVM offers a lower test accuracy compared to RF, but the results are excellent. The gap between the train and test accuracy indicates that model generalizes well and has less overfitting.
- **Efficiency:** SVM can be computational expensive, when trained with RBF kernel especially. It might be slower compared to LR or KNN on larger dataset.

7.3. K-Nearest Neighbors (KNN):

- Train Accuracy: 98.02%
- Test Accuracy: 94.74%
- **Explanation:** KNN has a comparatively high test accuracy, but there is drop in train accuracy. It shows that KNN does not capture all the instances of the data well like other models, resulting in lower performance.
- **Efficiency:** KNN is computationally expensive during prediction. It needs calculating distances of all training samples making it less effective for large datasets.

7.4. Logistic Regression:

- Train Accuracy: 98.68%
- Test Accuracy: 97.37
- **Explanation:** Logistic Regression has very similar training and testing accuracy for both test and train sets. The less difference suggests that model is generalizing well.
- **Efficiency:** Logistic Regression is computationally efficient, with smaller data. It is an ideal choice when interpretability and speed is needed.

7.5. Summary of Findings

Logistic Regression stands out as an optimal choice due to the balance between

accuracy and efficiency. While other models achieve high accuracy, they are computationally more expensive. LR maintains excellent accuracy while offers simplicity, quicker training times and low resource consumption making it an excellent choice for the project.

8. Conclusion on the Data Science Solution

In the report, we have applied multiple machine learning models to the Breast Cancer Wisconsin dataset to predict the diagnosis as either benign or malignant. Among the models tested LR,RF,SVM and KNN. The Logistic Regression has been considered as an optimal choice. However, the lack of the data made a problem for the models like RF that provided 100% train accuracy meaning that it overfitted. The solution can be further explored by testing the model in the real-world scenario to see how they perform and to make a genuine remark on them. However, the results suggest that LR is a strong candidate for the prediction. The code has not been explored to the extend required and will be explained in the presentation.

9. Appendices

Appendix A

Results for the Descriptive Analytics and Pre-processing

```
Variables DataFrame Head:
         name     role         type demographic description units  \
0          ID       ID  Categorical        None        None  None
1   Diagnosis   Target  Categorical        None        None  None
2     radius1  Feature   Continuous        None        None  None
3    texture1  Feature   Continuous        None        None  None
4  perimeter1  Feature   Continuous        None        None  None

  missing_values
0             no
1             no
2             no
3             no
4             no
```

```
Last few rows of the Dataset
     Index  ID  Diagnosis  radius1  texture1  perimeter1   area1  smoothness1  \
564    564 NaN        NaN    21.56     22.39      142.00  1479.0      0.11100
565    565 NaN        NaN    20.13     28.25      131.20  1261.0      0.09780
566    566 NaN        NaN    16.60     28.08      108.30   858.1      0.08455
567    567 NaN        NaN    20.60     29.33      140.10  1265.0      0.11780
568    568 NaN        NaN     7.76     24.54       47.92   181.0      0.05263

     compactness1  concavity1  ...  texture3  perimeter3   area3  smoothness3  \
564       0.11590     0.24390  ...     26.40      166.10  2027.0      0.14100
565       0.10340     0.14400  ...     38.25      155.00  1731.0      0.11660
566       0.10230     0.09251  ...     34.12      126.70  1124.0      0.11390
567       0.27700     0.35140  ...     39.42      184.60  1821.0      0.16500
568       0.04362     0.00000  ...     30.37       59.16   268.6      0.08996

     compactness3  concavity3  concave_points3  symmetry3  fractal_dimension3  \
564       0.21130      0.4107           0.2216     0.2060             0.07115
565       0.19220      0.3215           0.1628     0.2572             0.06637
566       0.30940      0.3403           0.1418     0.2218             0.07820
567       0.86810      0.9387           0.2650     0.4087             0.12400
568       0.06444      0.0000           0.0000     0.2871             0.07039
```

```
[5 rows x 34 columns]
Number of Benign and Malignant cases
target
B    357
M    212
dtype: int64
Description of the dataset
            Index   ID  Diagnosis     radius1    texture1  perimeter1  \
count  569.000000  0.0        0.0  569.000000  569.000000  569.000000
mean   284.000000  NaN        NaN   14.127292   19.289649   91.969033
std    164.400426  NaN        NaN    3.524049    4.301036   24.298981
min      0.000000  NaN        NaN    6.981000    9.710000   43.790000
25%    142.000000  NaN        NaN   11.700000   16.170000   75.170000
50%    284.000000  NaN        NaN   13.370000   18.840000   86.240000
75%    426.000000  NaN        NaN   15.780000   21.800000  104.100000
max    568.000000  NaN        NaN   28.110000   39.280000  188.500000

             area1  smoothness1  compactness1  concavity1  ...     radius3  \
count   569.000000   569.000000    569.000000  569.000000  ...  569.000000
mean    654.889104     0.096360      0.104341    0.088799  ...   16.269190
std     351.914129     0.014064      0.052813    0.079720  ...    4.833242
min     143.500000     0.052630      0.019380    0.000000  ...    7.930000
25%     420.300000     0.086370      0.064920    0.029560  ...   13.010000
50%     551.100000     0.095870      0.092630    0.061540  ...   14.970000
75%     782.700000     0.105300      0.130400    0.130700  ...   18.790000
max    2501.000000     0.163400      0.345400    0.426800  ...   36.040000
```

Further exploration is provided in the link:
https://drive.google.com/file/d/1gp0kPB42YqP922sml5MdklBTSbWwFQn8/view?usp=drivesdk

10. References

- Sun, Y.S., Zhao, Z., Yang, Z.N., Xu, F., Lu, H.J., Zhu, Z.Y., Shi, W., Jiang, J., Yao, P.P. and Zhu, H.P., 2017. Risk factors and preventions of breast

cancer. *International journal of biological sciences*, *13*(11), p.1387.

- World Health Organization (WHO) (n.d.) *Breast cancer*. Available at: https://www.who.int/news-room/fact-sheets/detail/breast-cancer (Accessed: 3 January 2025).
- Yue, W., Wang, Z., Chen, H., Payne, A. and Liu, X., 2018. Machine learning with applications in breast cancer diagnosis and prognosis. *Designs*, *2*(2), p.13.
- Wolberg, W., Mangasarian, O., Street, N., and Street, W. (1993) *Breast Cancer Wisconsin (Diagnostic) [Dataset].* UCI Machine Learning Repository. Available at: https://doi.org/10.24432/C5DW2B (Accessed: 3 January 2025).
- Google Developers. (2024). *Classification: Accuracy, Precision, and Recall*. Retrieved January 5, 2025, from https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall
- Evidently AI. (2024). *Classification Metrics: Accuracy, Precision, Recall*. Retrieved January 5, 2025, from https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall#:~:text=Precision%20is%20a%20metric%20that,both%20true%20and%20false%20positives)
- Serokell. (2020). *A Guide to F1 Score: What It Is, How to Calculate It, and When to Use It*. Retrieved January 5, 2025, from https://serokell.io/blog/a-guide-to-f1-score
- Sarang, Narkhede (2018). *Understanding AUC-ROC Curve*. Towards Data Science. Retrieved January 5, 2025, from https://towardsdatascience.com/understanding-auc-roc-curve-68b2303cc9c5
- diagrams.net. (2025). *diagrams.net*. [online] Available at: https://app.diagrams.net/ [Accessed 5 January 2025].
- GeeksforGeeks, 2024. *Confusion Matrix in Machine Learning*. [online] Available at: https://www.geeksforgeeks.org/confusion-matrix-machine-learning/ [Accessed 5 January 2025].
- IBM, n.d. *What is Logistic Regression?* [online] Available at: https://www.ibm.com/think/topics/logistic-regression [Accessed 5 January

2025].

- GeeksforGeeks, 2024. *Support Vector Machine Algorithm*. [online] Available at: https://www.geeksforgeeks.org/support-vector-machine-algorithm/ [Accessed 5 January 2025].
- Scikit-learn, 2024. *RandomForestClassifier* [online] Available at: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html [Accessed 5 January 2025].
- W3Schools, n.d. *Python Machine Learning - K-nearest neighbors (KNN)* [online] Available at: https://www.w3schools.com/python/python_ml_knn.asp [Accessed 5 January 2025].
- W3Schools, 2025. *Pandas Introduction* . [online] Available at: https://www.w3schools.com/python/pandas/pandas_intro.asp [Accessed 5 January 2025].
- Waskom, M.L., 2024. *Seaborn: Statistical data visualization*. [online] Available at: https://seaborn.pydata.org/ [Accessed 5 January 2025].
- Matplotlib, 2024. *Matplotlib: Visualization with Python*. [online] Available at: https://matplotlib.org/ [Accessed 5 January 2025].
- Bigelow, S.J., 2025. *What is NumPy? Explaining how it works in Python*. [online] TechTarget. Available at: https://www.techtarget.com/whatis/definition/What-is-NumPy-Explaining-how-it-works-in-Python [Accessed 5 January 2025].
- Jain, K., 2024. *Scikit-learn (sklearn) in Python – the most important Machine Learning tool I learnt last year!*. [online] Analytics Vidhya. Available at: https://www.analyticsvidhya.com/blog/2015/01/scikit-learn-python-machine-learning-tool/ [Accessed 5 January 2025].
- Johari, A., 2019. What is Python SciPy and How to use it? [online] Medium. Available at: https://medium.com/edureka/scipy-tutorial-38723361ba4b#:~:text=SciPy%20is%20an%20open%2Dsource,range%20of%20high%2Dlevel%20commands. [Accessed 5 January 2025].
- Chumbar, S. (2023) 'The CRISP-DM Process: A Comprehensive Guide'. *Medium*. Available at: https://medium.com/@shawn.chumbar/the-crisp-dm-

process-a-comprehensive-guide-4d893aecb151 (Accessed: 5 January 2025).